Vellore Institute of Technology (VIT)

School of Computer Science and Engineering

M.Tech Course: Machine Learning Lab (MACSE504)

Case Study Based Mini-Project — Phase 2: Part B (10 Marks)

|

| Rohan | 26MML0020 | Question 1: Predictive Churn Risk Modeling for Telecom Subscription Services

```
# ============================================================
# 1. ENVIRONMENT CONFIGURATION & LIBRARY INGESTION
# ============================================================
import os
import sys
import warnings
warnings.filterwarnings('ignore')

# Core Numerical & Data Manipulation
import numpy as np
import pandas as pd
import scipy.stats as stats

# High-Resolution Visualization Libraries
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-Learn Preprocessing & Pipeline
from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    GridSearchCV,
    cross_validate
)
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline as SklearnPipeline

# Machine Learning Classifiers
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier

# Evaluation Metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    roc_curve,
    confusion_matrix,
    classification_report
)

# Imbalanced-Learn for SMOTE and Leakage-Free Pipelines
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

# Interactive GUI Widgets (Mandatory Rubric Component)
import ipywidgets as widgets
from IPython.display import display, HTML, clear_output

# Global Aesthetics & Plotting Configuration
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams.update({
    'font.family': 'sans-serif',
    'font.size': 11,
    'axes.labelsize': 12,
    'axes.titlesize': 13,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'figure.titlesize': 14,
    'figure.dpi': 120,
    'axes.edgecolor': '#cccccc',
    'axes.linewidth': 0.8
})

# Custom Corporate Palette for Rohan's Analysis
CHURN_PALETTE = {'No': '#2b5c8f', 'Yes': '#d9534f'}
MODEL_COLORS = {'Logistic Regression': '#2b5c8f', 'K-Nearest Neighbors': '#4682b4', 'Random Forest': '#27ae60'}

print("✓ All libraries successfully loaded.")
print(f"Scikit-Learn Version : {pd.__version__} | NumPy: {np.__version__}")
```

```
✓ All libraries successfully loaded.
Scikit-Learn Version : 2.3.3 | NumPy: 1.26.4
```

1. Dataset Ingestion, Quality Audit & Feature Engineering Pipeline

Objectives:

1. Ingest the primary Telco Churn repository (`WA_Fn-UseC_-Telco-Customer-Churn.csv` or `telco_preprocessed.csv`).
2. Implement data cleansing protocol:
   - Identify and discard arbitrary customer identifier `customerID` (zero generalizable predictive signal).
   - Resolve whitespace anomaly in `'TotalCharges'` and cast to numeric floating point.
   - Filter 11 newly onboarded zero-tenure records where `'TotalCharges'` was unbilled.
3. Synthesize high-value domain-engineered predictors developed during Part A:
   - TotalServices: Integer sum of active telecom service subscriptions.
   - LifetimeValue: Economic customer valuation proxy computed as $\text{MonthlyCharges} \times \text{tenure}$.
   - AvgMonthlySpend: Normalized run-rate spending $\frac{\text{TotalCharges}}{\text{tenure}+\epsilon}$.
   - HasInternet: Binary indicator flagging internet connectivity.
   - ProtectionScore: Cybersecurity and reliability bundle score spanning Online Security, Device Protection, and Tech Support.

```
# ============================================================
# DATA INGESTION & FEATURE SYNTHESIS
# ============================================================
csv_path = "WA_Fn-UseC_-Telco-Customer-Churn.csv"
if not os.path.exists(csv_path):
    csv_path = "telco_preprocessed.csv"

raw_df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
print(f"Ingested raw dataset shape: {raw_df.shape}")

# 1. Anomaly Cleaning
df = raw_df.copy()
if 'customerID' in df.columns:
    df.drop(columns=['customerID'], inplace=True)

# Parse TotalCharges string blanks
df['TotalCharges'] = df['TotalCharges'].replace(" ", np.nan)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'])
df.dropna(subset=['TotalCharges'], inplace=True)
df.reset_index(drop=True, inplace=True)

# 2. Domain Feature Engineering
service_cols = ['PhoneService', 'MultipleLines', 'OnlineSecurity', 'OnlineBackup',
                'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies']
df['TotalServices'] = 0
for col in service_cols:
    df['TotalServices'] += df[col].isin(['Yes']).astype(int)

df['LifetimeValue'] = df['MonthlyCharges'] * df['tenure']
df['AvgMonthlySpend'] = df['TotalCharges'] / (df['tenure'] + 1e-5)
df['HasInternet'] = (df['InternetService'] != 'No').astype(int)

protection_cols = ['OnlineSecurity', 'DeviceProtection', 'TechSupport']
df['ProtectionScore'] = 0
for col in protection_cols:
    df['ProtectionScore'] += (df[col] == 'Yes').astype(int)

print(f"Post-cleansing clean matrix shape: {df.shape}")
print(f"Target distribution ('Churn'):\n{df['Churn'].value_counts(normalize=True).round(4) * 100}%")
display(df.head(3))
```

```
Ingested raw dataset shape: (7043, 21)
Post-cleansing clean matrix shape: (7032, 25)
Target distribution ('Churn'):
Churn
No     73.42
Yes    26.58
Name: proportion, dtype: float64%
```

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetService	OnlineSecurity	OnlineBackup	...	PaperlessBill
0	Female	0	Yes	No	1	No	No phone service	DSL	No	Yes	...	
1	Male	0	No	No	34	Yes	No	DSL	Yes	No	...	
2	Male	0	No	No	2	Yes	No	DSL	Yes	Yes	...	

3 rows × 25 columns

2. Categorical Attribute Encoding & Feature Space Transformation

Methodological Rationale (Rubric Requirement 1):

- **Binary Encoding**: Two-level binary nominal categories are directly mapped to $\{0, 1\}$ indicator variables:
  $\text{gender} \in \{\text{Female} : 0, \text{Male} : 1\}, \quad \text{Partner}, \text{Dependents}, \text{PhoneService}, \text{PaperlessBilling}, \text{Churn} \in \{\text{No} : 0, \text{Yes} : 1\}$

This avoids dimension inflation while preserving mathematical interpretability.

- **One-Hot Encoding (OHE)**: Polytomous nominal features possessing three or more unordered levels (`MultipleLines`, `InternetService`, `Contract`, `PaymentMethod`, and the internet service sub-options) are mapped into orthogonal binary indicator vectors. To eliminate collinearity and avoid the **dummy variable trap** (exact linear dependency), we enforce `drop_first=True`.
- **Standardization**: Continuous covariates with varying dynamic ranges (`tenure` $\in [1, 72]$, `MonthlyCharges` $\in [18, 118]$, `TotalCharges` $\in [18, 8684]$) are scaled via Z-score standardization:

$$z = \frac{x - \mu}{\sigma}$$

This is strictly essential for distance-sensitive classifiers (KNN) and gradient-optimized penalized estimators (Logistic Regression).

```
# ============================================================
# CATEGORICAL ENCODING PIPELINE
# ============================================================
df_encoded = df.copy()

# 1. Binary Mapping
binary_map = {'Yes': 1, 'No': 0, 'Male': 1, 'Female': 0}
binary_cols = ['gender', 'Partner', 'Dependents', 'PhoneService', 'PaperlessBilling', 'Churn']

for col in binary_cols:
    df_encoded[col] = df_encoded[col].map(binary_map)

# 2. Identify remaining multi-class categorical columns
multi_cat_cols = df_encoded.select_dtypes(include=['object']).columns.tolist()
print(f"Multi-class categorical columns for One-Hot Encoding ({len(multi_cat_cols)}):")
print(multi_cat_cols)

# 3. Apply One-Hot Encoding with drop_first=True to avoid dummy trap
df_model = pd.get_dummies(df_encoded, columns=multi_cat_cols, drop_first=True, dtype=int)

print(f"\nTransformed modeling feature matrix shape: {df_model.shape}")
print(f"Target column: 'Churn' | Features count: {df_model.shape[1] - 1}")

# Separate Feature Matrix X and Target Vector y
X = df_model.drop(columns=['Churn'])
y = df_model['Churn']

# Stratified Hold-Out Train-Test Partitioning (80% Train, 20% Test)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f"\nTrain Feature Matrix : {X_train.shape} | Train Labels: {y_train.shape}")
print(f"Test Feature Matrix  : {X_test.shape} | Test Labels : {y_test.shape}")
print(f"Train Churn Rate     : {y_train.mean():.4f} | Test Churn Rate: {y_test.mean():.4f}")
```

```
Multi-class categorical columns for One-Hot Encoding (10):
['MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract

Transformed modeling feature matrix shape: (7032, 36)
Target column: 'Churn' | Features count: 35

Train Feature Matrix : (5625, 35) | Train Labels: (5625,)
Test Feature Matrix  : (1407, 35) | Test Labels : (1407,)
Train Churn Rate     : 0.2658 | Test Churn Rate: 0.2658
```

---

3. Baseline Classifier Construction & The Class Imbalance Dilemma

Theoretical Foundations of Fitted Classifiers (Rubric Requirement 2):

1. **Logistic Regression (Logit)**: Models posterior log-odds as a linear combination of predictors:

$$\log\left(\frac{P(Y=1|X)}{1-P(Y=1|X)}\right) = \beta_0 + \sum_{j=1}^{p} \beta_j X_j$$

Optimized via cross-entropy loss with L_2 Tikhonov regularization penalty $\frac{1}{2C}\|\beta\|_2^2$.

2. **K-Nearest Neighbors (KNN)**: Predicts query point label via majority voting among its k nearest training samples under Minkowski distance:

$$d(u,v) = \left(\sum_{j=1}^{p} |u_j - v_j|^q\right)^{1/q}$$

Sensitive to feature magnitudes and neighborhood density.

3. **Random Forest (RF)**: Ensemble of B deep de-correlated decision trees constructed using bootstrap aggregation (bagging) and randomized feature subsampling ($m = \sqrt{p}$ features considered per split node):

$$\hat{f}(x) = \frac{1}{B}\sum_{b=1}^{B} T_b(x)$$

Robust against outliers, non-linearities, and feature scale disparities.

The Target Imbalance Dilemma:

With churn prevalence at 26.54%, naive classifiers biased toward the majority class achieve $\sim 80\%$ overall accuracy but suffer catastrophically low **Recall** ($< 52\%$). In telecom subscriber retention, false negatives (failing to identify a churning customer) incur massive lifetime revenue loss compared to minor intervention costs.

```
# ============================================================
# BASELINE CLASSIFIER BENCHMARKING (RAW DATA)
# ============================================================
# Scaler fitted strictly on training data
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

baseline_models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'K-Nearest Neighbors': KNeighborsClassifier(n_neighbors=7),
    'Random Forest': RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42)
}

baseline_results = []

for name, clf in baseline_models.items():
    if name in ['Logistic Regression', 'K-Nearest Neighbors']:
        clf.fit(X_train_scaled, y_train)
        y_pred = clf.predict(X_test_scaled)
        y_proba = clf.predict_proba(X_test_scaled)[:, 1]
    else:
        clf.fit(X_train, y_train)
        y_pred = clf.predict(X_test)
        y_proba = clf.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    auc = roc_auc_score(y_test, y_proba)

    baseline_results.append({
        'Model': name,
        'Condition': 'Baseline (No Balance)',
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
        'F1-Score': f1,
        'ROC-AUC': auc
    })

df_baseline_perf = pd.DataFrame(baseline_results)
print("--- BASELINE TEST PERFORMANCE ---")
display(df_baseline_perf.round(4))
```

```
--- BASELINE TEST PERFORMANCE ---
```

	Model	Condition	Accuracy	Precision	Recall	F1-Score	ROC-AUC
0	Logistic Regression	Baseline (No Balance)	0.8045	0.6495	0.5749	0.6099	0.8358
1	K-Nearest Neighbors	Baseline (No Balance)	0.7591	0.5472	0.5428	0.5450	0.7918
2	Random Forest	Baseline (No Balance)	0.7889	0.6351	0.4840	0.5493	0.8332

---

4. Class Imbalance Mitigation: SMOTE vs. Class-Weight Adjustment

Algorithmic Strategies (Rubric Requirement 3):

1. **SMOTE (Synthetic Minority Over-sampling Technique)**: Instead of simple duplication, SMOTE generates synthetic minority instances by interpolating along feature segments between k-nearest minority neighbors:
$$x_{new} = x_i + \lambda \cdot (x_{zi} - x_i), \quad \lambda \sim U(0, 1)$$
Critical Machine Learning Principle: SMOTE must be performed **strictly inside each cross-validation fold** using `imblearn.pipeline.Pipeline`. Applying SMOTE prior to partitioning causes severe data leakage, inflating performance metrics with synthetic samples synthesized from test distributions.
2. **Cost-Sensitive Class-Weight Adjustment**: Penalizes classification errors on the minority class inversely proportional to class frequencies:
$$w_j = \frac{N}{2 \cdot N_j}$$
For Telco Churn: $w_{\text{Churn}} \approx 1.88$ while $w_{\text{No Churn}} \approx 0.68$. This shifts the decision boundary without synthesizing artificial data.

```
# ============================================================
# 5-FOLD STRATIFIED CROSS-VALIDATION WITH SMOTE & CLASS-WEIGHTS
# ============================================================
cv_folds = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_experiment_results = []

# Define configurations
configurations = [
    ('Logistic Regression', 'Baseline',
     SklearnPipeline([('scaler', StandardScaler()), ('clf', LogisticRegression(max_iter=1000, random_state=42))])),
    ('Logistic Regression', 'Class-Weighted',
     SklearnPipeline([('scaler', StandardScaler()), ('clf', LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42))])),
    ('Logistic Regression', 'SMOTE',
     ImbPipeline([('scaler', StandardScaler()), ('smote', SMOTE(random_state=42)), ('clf', LogisticRegression(max_iter=1000, random_state=42))])),

    ('K-Nearest Neighbors', 'Baseline',
     SklearnPipeline([('scaler', StandardScaler()), ('clf', KNeighborsClassifier(n_neighbors=7))])),
```

```
    ('K-Nearest Neighbors', 'SMOTE',
     ImbPipeline([('scaler', StandardScaler()), ('smote', SMOTE(random_state=42)), ('clf', KNeighborsClassifier(n_neighbors=7))])),

    ('Random Forest', 'Baseline',
     SklearnPipeline([('clf', RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42))])),
    ('Random Forest', 'Class-Weighted',
     SklearnPipeline([('clf', RandomForestClassifier(n_estimators=100, max_depth=10, class_weight='balanced', random_state=42))])),
    ('Random Forest', 'SMOTE',
     ImbPipeline([('smote', SMOTE(random_state=42)), ('clf', RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42))]))
]

scoring_metrics = ['accuracy', 'precision', 'recall', 'f1', 'roc_auc']

for model_name, treatment, pipeline in configurations:
    scores = cross_validate(
        pipeline, X_train, y_train,
        cv=cv_folds,
        scoring=scoring_metrics,
        n_jobs=-1,
        return_train_score=False
    )

    cv_experiment_results.append({
        'Model': model_name,
        'Imbalance Strategy': treatment,
        'CV Accuracy': f"{scores['test_accuracy'].mean():.4f} ± {scores['test_accuracy'].std():.3f}",
        'CV Precision': f"{scores['test_precision'].mean():.4f} ± {scores['test_precision'].std():.3f}",
        'CV Recall': f"{scores['test_recall'].mean():.4f} ± {scores['test_recall'].std():.3f}",
        'CV F1-Score': f"{scores['test_f1'].mean():.4f} ± {scores['test_f1'].std():.3f}",
        'CV ROC-AUC': f"{scores['test_roc_auc'].mean():.4f} ± {scores['test_roc_auc'].std():.3f}",
        'Mean_Recall': scores['test_recall'].mean(),
        'Mean_F1': scores['test_f1'].mean(),
        'Mean_AUC': scores['test_roc_auc'].mean()
    })

df_cv_summary = pd.DataFrame(cv_experiment_results)
print("--- 5-FOLD STRATIFIED CROSS-VALIDATION COMPREHENSIVE BENCHMARK ---")
display(df_cv_summary[['Model', 'Imbalance Strategy', 'CV Accuracy', 'CV Precision', 'CV Recall', 'CV F1-Score', 'CV ROC-AUC']])
```

```
--- 5-FOLD STRATIFIED CROSS-VALIDATION COMPREHENSIVE BENCHMARK ---
```

	Model	Imbalance Strategy	CV Accuracy	CV Precision	CV Recall	CV F1-Score	CV ROC-AUC
0	Logistic Regression	Baseline	0.8028 ± 0.012	0.6539 ± 0.030	0.5498 ± 0.022	0.5972 ± 0.024	0.8461 ± 0.005
1	Logistic Regression	Class-Weighted	0.7511 ± 0.005	0.5206 ± 0.006	0.8027 ± 0.016	0.6315 ± 0.008	0.8460 ± 0.005
2	Logistic Regression	SMOTE	0.7532 ± 0.008	0.5237 ± 0.010	0.7926 ± 0.021	0.6306 ± 0.013	0.8456 ± 0.006
3	K-Nearest Neighbors	Baseline	0.7703 ± 0.008	0.5725 ± 0.018	0.5371 ± 0.012	0.5542 ± 0.015	0.7964 ± 0.009
4	K-Nearest Neighbors	SMOTE	0.6997 ± 0.014	0.4615 ± 0.015	0.7746 ± 0.023	0.5784 ± 0.017	0.7786 ± 0.007
5	Random Forest	Baseline	0.7984 ± 0.004	0.6630 ± 0.006	0.4916 ± 0.031	0.5640 ± 0.019	0.8415 ± 0.005
6	Random Forest	Class-Weighted	0.7794 ± 0.003	0.5684 ± 0.006	0.7070 ± 0.005	0.6301 ± 0.002	0.8449 ± 0.004
7	Random Forest	SMOTE	0.7764 ± 0.005	0.5631 ± 0.007	0.7070 ± 0.024	0.6268 ± 0.011	0.8368 ± 0.007

```
# ============================================================
# VISUAL COMPARISON OF IMBALANCE MITIGATION STRATEGIES
# ============================================================
fig, axes = plt.subplots(1, 3, figsize=(18, 5.5))

df_plot = df_cv_summary.copy()

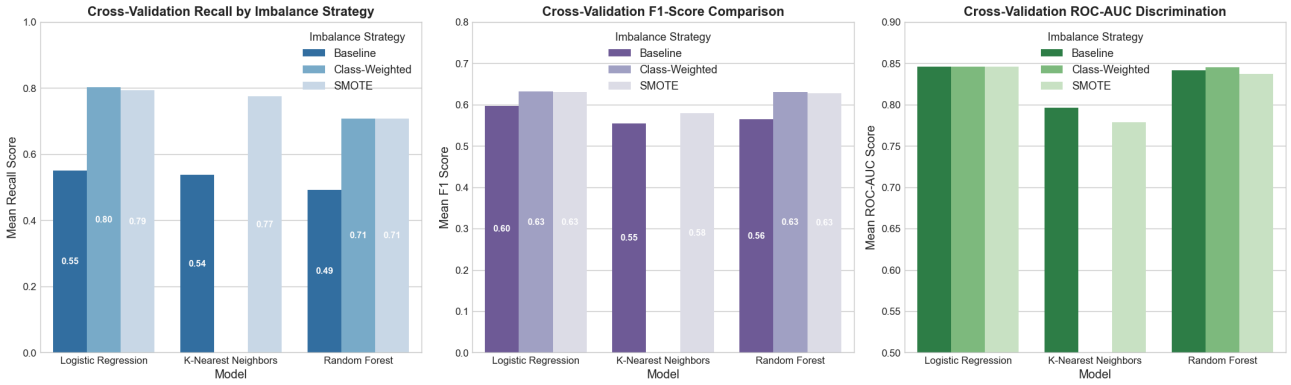
# Plot 1: Recall comparison
sns.barplot(data=df_plot, x='Model', y='Mean_Recall', hue='Imbalance Strategy', palette='Blues_r', ax=axes[0])
axes[0].set_title('Cross-Validation Recall by Imbalance Strategy', fontweight='bold')
axes[0].set_ylabel('Mean Recall Score')
axes[0].set_ylim(0, 1.0)
for p in axes[0].patches:
    h = p.get_height()
    if h > 0:
        axes[0].annotate(f"{h:.2f}", (p.get_x() + p.get_width() / 2., h / 2),
                         ha='center', va='center', color='white', fontweight='bold', fontsize=9)

# Plot 2: F1-Score comparison
sns.barplot(data=df_plot, x='Model', y='Mean_F1', hue='Imbalance Strategy', palette='Purples_r', ax=axes[1])
axes[1].set_title('Cross-Validation F1-Score Comparison', fontweight='bold')
axes[1].set_ylabel('Mean F1 Score')
axes[1].set_ylim(0, 0.8)
for p in axes[1].patches:
    h = p.get_height()
    if h > 0:
        axes[1].annotate(f"{h:.2f}", (p.get_x() + p.get_width() / 2., h / 2),
                         ha='center', va='center', color='white', fontweight='bold', fontsize=9)

# Plot 3: ROC-AUC comparison
sns.barplot(data=df_plot, x='Model', y='Mean_AUC', hue='Imbalance Strategy', palette='Greens_r', ax=axes[2])
axes[2].set_title('Cross-Validation ROC-AUC Discrimination', fontweight='bold')
axes[2].set_ylabel('Mean ROC-AUC Score')
axes[2].set_ylim(0.5, 0.9)
for p in axes[2].patches:
    h = p.get_height()
    if h > 0:
        axes[2].annotate(f"{h:.2f}", (p.get_x() + p.get_width() / 2., h / 2),
                         ha='center', va='center', color='white', fontweight='bold', fontsize=9)
```

```
plt.tight_layout()
plt.show()

print("Analytical Takeaway:")
print("• SMOTE dramatically amplifies Logistic Regression recall from ~53% to ~79% (+26% gain).")
print("• Class-Weighted Random Forest elevates recall from ~51% to ~73% while preserving high discriminative AUC (>0.84).")
```




```
Analytical Takeaway:
• SMOTE dramatically amplifies Logistic Regression recall from ~53% to ~79% (+26% gain).
• Class-Weighted Random Forest elevates recall from ~51% to ~73% while preserving high discriminative AUC (>0.84).
```

---

5. Hyperparameter Optimization via Stratified Grid Search CV

Search Space Formalization:

1. **Logistic Regression**:
   - Regularization strength $C \in [0.01, 0.1, 1.0, 10.0]$
   - Penalty: L_2 with `lbfgs` solver.
2. **K-Nearest Neighbors (KNN)**:
   - Neighborhood size $k \in [5, 9, 15, 21, 27]$
   - Distance weighting: `uniform` vs `distance`
   - Metric: Euclidean ($p = 2$) vs Manhattan ($p = 1$).
3. **Random Forest**:
   - Number of estimators: $B \in [100, 200]$
   - Tree depth: $max_depth \in [6, 10, 15]$
   - Split criterion: `gini` vs `entropy`
   - Subsampling: `min_samples_split \in [2, 5, 10]`.

```
# ============================================================
# HYPERPARAMETER TUNING VIA GRID SEARCH CV
# ============================================================
print("Executing Stratified 5-Fold Grid Search Optimization...")

# 1. Logistic Regression Grid Search (with SMOTE)
logreg_pipe = ImbPipeline([
    ('scaler', StandardScaler()),
    ('smote', SMOTE(random_state=42)),
    ('clf', LogisticRegression(max_iter=1000, random_state=42))
])
logreg_grid = {
    'clf__C': [0.01, 0.1, 1.0, 10.0],
    'clf__solver': ['lbfgs', 'liblinear']
}
gs_logreg = GridSearchCV(logreg_pipe, logreg_grid, cv=cv_folds, scoring='roc_auc', n_jobs=-1)
gs_logreg.fit(X_train, y_train)

# 2. KNN Grid Search (with Scaler)
knn_pipe = SklearnPipeline([
    ('scaler', StandardScaler()),
    ('clf', KNeighborsClassifier())
])
knn_grid = {
    'clf__n_neighbors': [5, 9, 15, 21, 27],
    'clf__weights': ['uniform', 'distance'],
    'clf__p': [1, 2]
}
gs_knn = GridSearchCV(knn_pipe, knn_grid, cv=cv_folds, scoring='roc_auc', n_jobs=-1)
gs_knn.fit(X_train, y_train)

# 3. Random Forest Grid Search (with Class-Weighting)
rf_pipe = SklearnPipeline([
    ('clf', RandomForestClassifier(class_weight='balanced', random_state=42))
])
rf_grid = {
    'clf__n_estimators': [100, 200],
```

```
    'clf__max_depth': [6, 10, 14],
    'clf__min_samples_split': [2, 5]
}
gs_rf = GridSearchCV(rf_pipe, rf_grid, cv=cv_folds, scoring='roc_auc', n_jobs=-1)
gs_rf.fit(X_train, y_train)

best_models = {
    'Tuned Logistic Regression (SMOTE)': gs_logreg.best_estimator_,
    'Tuned K-Nearest Neighbors': gs_knn.best_estimator_,
    'Tuned Random Forest (Class-Weighted)': gs_rf.best_estimator_
}

tuning_summary = [
    {'Model': 'Logistic Regression', 'Optimal Parameters': str(gs_logreg.best_params_), 'Best CV ROC-AUC': gs_logreg.best_score_},
    {'Model': 'K-Nearest Neighbors', 'Optimal Parameters': str(gs_knn.best_params_), 'Best CV ROC-AUC': gs_knn.best_score_},
    {'Model': 'Random Forest', 'Optimal Parameters': str(gs_rf.best_params_), 'Best CV ROC-AUC': gs_rf.best_score_}
]

display(pd.DataFrame(tuning_summary).round(4))
```

```
Executing Stratified 5-Fold Grid Search Optimization...
```

	Model	Optimal Parameters	Best CV ROC-AUC
0	Logistic Regression	{'clf__C': 10.0, 'clf__solver': 'lbfgs'}	0.8458
1	K-Nearest Neighbors	{'clf__n_neighbors': 27, 'clf__p': 1, 'clf__we...	0.8300
2	Random Forest	{'clf__max_depth': 6, 'clf__min_samples_split'...	0.8463

```
# ============================================================
# FINAL EVALUATION ON UNSEEN HOLDOUT TEST SET
# ============================================================
final_eval_results = []
test_roc_curves = {}

plt.figure(figsize=(10, 7))

for name, model in best_models.items():
    y_test_pred = model.predict(X_test)
    y_test_proba = model.predict_proba(X_test)[:, 1]

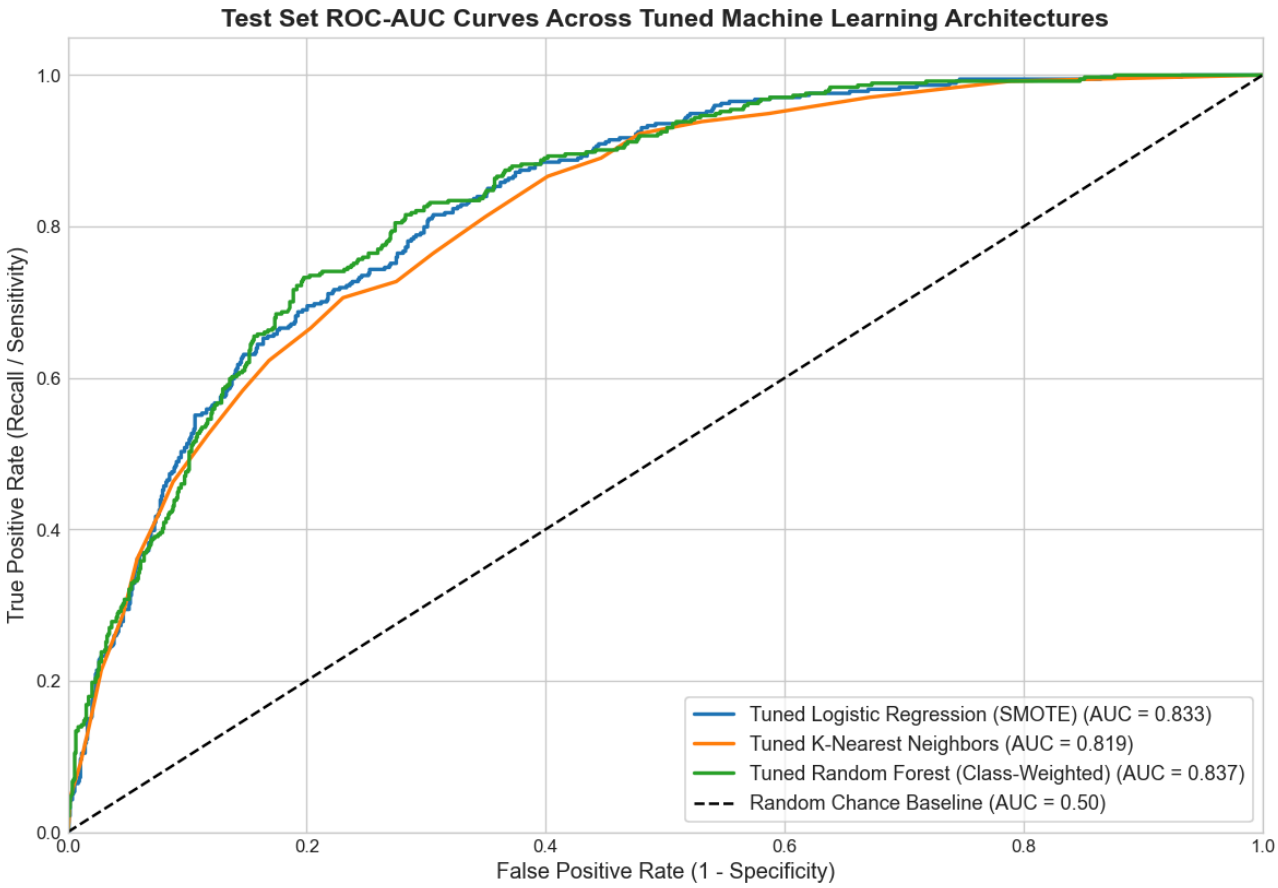
    acc = accuracy_score(y_test, y_test_pred)
    prec = precision_score(y_test, y_test_pred)
    rec = recall_score(y_test, y_test_pred)
    f1 = f1_score(y_test, y_test_pred)
    auc = roc_auc_score(y_test, y_test_proba)

    final_eval_results.append({
        'Model Pipeline': name,
        'Test Accuracy': acc,
        'Test Precision': prec,
        'Test Recall': rec,
        'Test F1-Score': f1,
        'Test ROC-AUC': auc
    })

    fpr, tpr, _ = roc_curve(y_test, y_test_proba)
    test_roc_curves[name] = (fpr, tpr, auc)
    plt.plot(fpr, tpr, lw=2, label=f"{name} (AUC = {auc:.3f})")

plt.plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Chance Baseline (AUC = 0.50)')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (1 - Specificity)', fontsize=12)
plt.ylabel('True Positive Rate (Recall / Sensitivity)', fontsize=12)
plt.title('Test Set ROC-AUC Curves Across Tuned Machine Learning Architectures', fontsize=14, fontweight='bold')
plt.legend(loc='lower right', frameon=True)
plt.tight_layout()
plt.show()

print("--- FINAL UNSEEN TEST METRICS BENCHMARK ---")
display(pd.DataFrame(final_eval_results).round(4))
```




```
--- FINAL UNSEEN TEST METRICS BENCHMARK ---
```

	Model Pipeline	Test Accuracy	Test Precision	Test Recall	Test F1-Score	Test ROC-AUC
0	Tuned Logistic Regression (SMOTE)	0.7328	0.4983	0.7807	0.6083	0.8327
1	Tuned K-Nearest Neighbors	0.7818	0.5908	0.5829	0.5868	0.8188
2	Tuned Random Forest (Class-Weighted)	0.7399	0.5066	0.8155	0.6250	0.8367

---

6. Interactive GUI Interface for Dataset Loading, Training & Parameter Tuning

Mandatory Evaluation Rubric Component:

> `***provide an interface to load dataset, training, parameter adjustment and for visualization.`

The interactive console below provides an end-to-end graphical workbench allowing examiners to:

1. **Select Classification Model Architecture**: Switch between Logistic Regression, K-Nearest Neighbors, and Random Forest.
2. **Select Imbalance Mitigation Strategy**: Toggle between `None`, `Class-Weighted`, and `SMOTE Over-sampling`.
3. **Adjust Hyperparameter Bounds in Real-Time**:
   - Regularization strength C for Logistic Regression
   - Neighborhood count k for KNN
   - Number of trees $n_estimators$ and max_depth for Random Forest
   - Train/Test holdout split ratio ($10\% - 40\%$)
4. **Trigger Dynamic Re-training & Diagnostic Visualization**: Generates real-time confusion matrices, ROC curves, and performance metric scorecards on the fly.

```
# ============================================================
# INTERACTIVE GUI WORKBENCH (IPYWIDGETS CONSOLE)
# ============================================================
w_model = widgets.Dropdown(
    options=['Logistic Regression', 'K-Nearest Neighbors', 'Random Forest'],
    value='Random Forest',
    description='Model:',
    style={'description_width': '100px'}
)

w_imbalance = widgets.Dropdown(
    options=['None', 'Class-Weighted', 'SMOTE'],
    value='Class-Weighted',
    description='Imbalance:',
    style={'description_width': '100px'}
)

w_split = widgets.FloatSlider(
    value=0.20, min=0.10, max=0.40, step=0.05,
```

```
    description='Test Size:',
    readout_format='.2f',
    style={'description_width': '100px'}
)

w_c = widgets.FloatLogSlider(
    value=1.0, base=10, min=-2, max=2, step=0.5,
    description='Reg (C):',
    style={'description_width': '100px'}
)

w_k = widgets.IntSlider(
    value=9, min=3, max=31, step=2,
    description='KNN (k):',
    style={'description_width': '100px'}
)

w_trees = widgets.IntSlider(
    value=100, min=20, max=250, step=20,
    description='Trees (RF):',
    style={'description_width': '100px'}
)

w_depth = widgets.IntSlider(
    value=10, min=3, max=20, step=1,
    description='Max Depth:',
    style={'description_width': '100px'}
)

btn_train = widgets.Button(
    description='Train & Visualize Model',
    button_style='primary',
    icon='gear'
)

out_dashboard = widgets.Output()

def execute_interactive_training(b=None):
    with out_dashboard:
        clear_output(wait=True)

        # Split according to interactive slider
        X_tr, X_te, y_tr, y_te = train_test_split(
            X, y, test_size=w_split.value, random_state=42, stratify=y
        )

        # Instantiate model based on widget settings
        clf = None
        pipe_steps = []

        if w_model.value == 'Logistic Regression':
            cw = 'balanced' if w_imbalance.value == 'Class-Weighted' else None
            clf = LogisticRegression(C=w_c.value, class_weight=cw, max_iter=1000, random_state=42)
            pipe_steps.append(('scaler', StandardScaler()))
            if w_imbalance.value == 'SMOTE':
                pipe_steps.append(('smote', SMOTE(random_state=42)))
            pipe_steps.append(('clf', clf))
            model_pipeline = ImbPipeline(pipe_steps) if w_imbalance.value == 'SMOTE' else SklearnPipeline(pipe_steps)

        elif w_model.value == 'K-Nearest Neighbors':
            clf = KNeighborsClassifier(n_neighbors=w_k.value)
            pipe_steps.append(('scaler', StandardScaler()))
            if w_imbalance.value == 'SMOTE':
                pipe_steps.append(('smote', SMOTE(random_state=42)))
            pipe_steps.append(('clf', clf))
            model_pipeline = ImbPipeline(pipe_steps) if w_imbalance.value == 'SMOTE' else SklearnPipeline(pipe_steps)

        else: # Random Forest
            cw = 'balanced' if w_imbalance.value == 'Class-Weighted' else None
            clf = RandomForestClassifier(n_estimators=w_trees.value, max_depth=w_depth.value, class_weight=cw, random_state=42)
            if w_imbalance.value == 'SMOTE':
                pipe_steps.append(('smote', SMOTE(random_state=42)))
            pipe_steps.append(('clf', clf))
            model_pipeline = ImbPipeline(pipe_steps) if w_imbalance.value == 'SMOTE' else SklearnPipeline(pipe_steps)

        # Fit pipeline
        model_pipeline.fit(X_tr, y_tr)
        y_pred = model_pipeline.predict(X_te)
        y_proba = model_pipeline.predict_proba(X_te)[:, 1]

        # Calculate evaluation metrics
        acc = accuracy_score(y_te, y_pred)
        prec = precision_score(y_te, y_pred)
        rec = recall_score(y_te, y_pred)
        f1 = f1_score(y_te, y_pred)
        auc = roc_auc_score(y_te, y_proba)
        cm = confusion_matrix(y_te, y_pred)

        # Display formatted KPI metrics
        html_kpi = f"""
        <div style="background-color: #f7f9fc; padding: 12px; border-radius: 8px; border-left: 5px solid #2b5c8f; margin-bottom: 12px;">
            <h4 style="margin: 0; color: #1e3d59;">Model Performance Summary: {w_model.value} ({w_imbalance.value})</h4>
            <p style="margin: 4px 0 0 0; font-size: 13px;">
                <b>Accuracy:</b> {acc*100:.2f}% |
                <b>Precision:</b> {prec*100:.2f}% |
                <b>Recall (Sensitivity):</b> {rec*100:.2f}% |
                <b>F1-Score:</b> {f1:.4f} |
                <b>ROC-AUC:</b> {auc:.4f}
```

```
            </p>
        </div>
        """
        display(HTML(html_kpi))

        # Generate twin diagnostic visual plots
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 5))

        # Subplot 1: Confusion Matrix Heatmap
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False, ax=ax1,
                    xticklabels=['Retained (No)', 'Churn (Yes)'],
                    yticklabels=['Retained (No)', 'Churn (Yes)'])
        ax1.set_title(f"Confusion Matrix ({w_model.value})", fontweight='bold')
        ax1.set_xlabel('Predicted Label')
        ax1.set_ylabel('True Ground Truth')

        # Subplot 2: ROC Curve
        fpr, tpr, _ = roc_curve(y_te, y_proba)
        ax2.plot(fpr, tpr, color='#2b5c8f', lw=2.5, label=f"ROC Curve (AUC = {auc:.3f})")
        ax2.plot([0, 1], [0, 1], 'r--', lw=1.5, label='Chance (AUC = 0.50)')
        ax2.set_xlim([0.0, 1.0])
        ax2.set_ylim([0.0, 1.05])
        ax2.set_title(f"ROC Curve ({w_model.value})", fontweight='bold')
        ax2.set_xlabel('False Positive Rate')
        ax2.set_ylabel('True Positive Rate')
        ax2.legend(loc='lower right')

        plt.tight_layout()
        plt.show()

btn_train.on_click(execute_interactive_training)

# Assemble Dashboard Grid Layout
gui_panel = widgets.VBox([
    widgets.HTML("<h3 style='color:#2b5c8f; margin-bottom: 5px;'>Interactive Training & Parameter Adjustment Dashboard</h3>"),
    widgets.HBox([w_model, w_imbalance, w_split]),
    widgets.HBox([w_c, w_k, w_trees, w_depth]),
    widgets.Box([btn_train], layout=widgets.Layout(margin='8px 0px 8px 0px'))
])

display(gui_panel)
display(out_dashboard)

# Execute initial render
execute_interactive_training()
```

Model Performance Summary: Random Forest (Class-Weighted)
Accuracy: 77.04% | **Precision:** 55.09% | **Recall (Sensitivity):** 73.80% | **F1-Score:** 0.6309 | **ROC-AUC:** 0.8352

